

Prime Stack Test Plan

Team: Prime Stack
Application under test: Frost Inventory and Shopping System
Report date: 19 May 2026

1. Introduction

This test plan describes the validation, verification, performance, usability, and security testing strategy for the Frost Inventory and Shopping System. The application is a MEAN-style web application with an Angular frontend, an Express backend API, MySQL persistence, session-based authentication, product and supplier administration, purchase history, and a client shopping/cart workflow.

2. Scope

In scope:

- User registration, login, logout, authentication status, and protected profile access.
- Product creation, listing, detailed listing, lookup, stock update, and deletion.
- Supplier creation, listing, lookup, update, and deletion.
- Purchase creation, purchase history retrieval, sorting, and purchased-product retrieval.
- Frontend layout, login/register validation, protected navigation, product catalog, cart behavior, purchase completion, profile viewing, and profile editing.
- API performance under normal, load, stress, and recovery conditions.
- Security testing using scripted k6 attacks and OWASP ZAP automated scanning.

Out of scope:

- Payment gateway integration, because no external payment service exists in the application.
- Production HTTPS, deployment infrastructure, and third-party identity provider testing.
- Browser compatibility beyond Chromium and Firefox unless requested by the instructor.

3. Test Objectives

- Verify that major user-facing and API workflows behave according to requirements.
- Confirm that input validation handles normal, boundary, duplicate, missing, and invalid data cases.
- Measure response time and error rate under increasing virtual-user load.
- Identify common web security weaknesses, especially injection, authorization, security headers, and information disclosure issues.
- Document the confirmed defects DEF-001 through DEF-010 and open usability findings DEF-011 through DEF-012 in a reproducible format with severity, priority, evidence, and suggested fixes.

4. Test Items

Area	Components / endpoints
User	/User/register , /User/login , /User/logout , /User/status , /User/profile , /User/users
Product	/Product/ , /Product/getALL , /Product/getByProductId/:id , /Product/getBySupplierId/:supplier , /Product/decreaseAmount/:id , /Product/:id
Supplier	/Supplier/ , /Supplier/:id

Area	Components / endpoints
Purchase	<code>/Purchase/create</code> , <code>/Purchase/getAll</code> , <code>/Purchase/byUserId/:userId</code> , <code>/Purchase/byUserInAsc/:userId</code> , <code>/Purchase/byUserInDesc/:userId</code> , <code>/Purchase/byPurchase/:purchaseId</code>
Frontend	Application shell, login/register forms, product catalog, cart, purchase flow, profile view/edit, admin/client navigation
Security	Express response headers, protected routes, SQL injection payloads, XSS payloads, mass assignment, oversized payloads, unauthorized user-list access, authenticated session/profile access

5. Test Approach

Functional testing uses automated Jest and Supertest tests against the Express API. Test cases cover normal paths, boundary values, missing data, duplicate data, invalid identifiers, and invalid query parameters.

Frontend/usability testing uses Playwright. These tests open the running Angular application in Chromium and Firefox, execute user-oriented scenarios, and save screenshots as evidence.

Performance testing uses k6. The load test ramps from 5 virtual users to 20 virtual users, then stress tests at 50 virtual users before ramping down to zero. The target thresholds are `p(95)<500ms` and error rate below 10%.

Security testing uses two approaches. First, a scripted k6 security test sends SQL injection, XSS, mass assignment, unauthenticated access, authenticated session/profile access, and oversized payload requests. Second, OWASP ZAP 2.17.0 scans `http://localhost:3000` and records passive/active findings.

Usability testing is based on representative user tasks for an admin and a client. The evaluation focuses on task completion, clarity of validation messages, navigation consistency, and whether users can complete product browsing and cart actions without confusion.

6. Test Environment

Item	Value
Operating environment	Local development machine
Backend	Node.js, Express 5.1.0
Frontend	Angular 20.1.x, Bootstrap 5.3.7
Database	MySQL, database name <code>frost</code>
Functional test tools	Jest, Supertest
Frontend/usability test tool	Playwright
Performance/security script tool	k6
Security scanner	ZAP by Checkmarx 2.17.0
Backend URL	<code>http://localhost:3000</code>
Frontend URL	<code>http://localhost:4200</code>

7. Entry Criteria

- Backend dependencies are installed and `.env` contains valid database/session configuration.
- MySQL is running and the `frost` database is available.
- Backend server can start on port `3000`.
- Frontend can start on port `4200`.

- Test data is available for products, suppliers, and at least one user.
- ZAP scans are performed only against the local application.

8. Exit Criteria

- All planned functional test cases have been executed or explicitly marked blocked.
- Performance test results are recorded against the defined thresholds.
- Security scan findings are documented in the defect report.
- Critical and high severity defects are either fixed or clearly accepted as known risks.
- Test execution evidence is stored in `docs/testing/evidence`.

9. Risks and Mitigation

Risk	Impact	Mitigation
Database state affects automated test repeatability	Tests may fail because duplicate or missing data exists	Tests clean up known test records before and after execution where possible
Tests close the shared database pool	Running all Jest suites together may cause connection reuse issues	Run suites separately if needed or refactor teardown into a global Jest setup
Local-only security scan may miss production configuration risks	Security coverage is incomplete for deployment	Document local scope and repeat scans after deployment configuration exists
Missing authentication on admin endpoints	Unauthorized data exposure	Log as a security defect and add middleware to sensitive routes
Mass assignment in registration	Any public user may become admin	Use field allow-listing and server-side role assignment
Stored XSS in user-controlled fields	Malicious script may execute when rendered	Validate input and escape output
Session user values are stored as objects instead of strings	Frontend may render <code>[object Object]</code> for profile data	Store primitive session fields and verify profile UI with Playwright
API response wrapper differs from frontend expectation	Profile may render blank after edit/refresh	Normalize <code>response.user</code> before assigning component state
Security headers are inconsistent	Browser-side protection is weaker	Add centralized security middleware such as Helmet

10. Roles and Responsibilities

Role	Responsibility
Test Manager	Maintains this plan, tracks schedule, reviews deliverables
Test Engineer	Designs and executes functional, usability, performance, and security tests
Programmer	Maintains automated tests, k6 scripts, ZAP setup, and environment configuration
Interface Designer / Usability Tester	Reviews UI workflows, performs usability scenarios, documents UI issues

11. Schedule

Week	Activity
3	Register group name and assign roles
4	Confirm application under test and scope
5-6	Prepare test plan and functional test cases
7-8	Implement and execute API/frontend automated tests
9-10	Execute performance and usability testing
11-12	Execute security testing and ZAP scan
13	Finalize execution report and defect report
14	Submit PDFs and present results

12. Deliverables

- PrimeStack_TestPlan.pdf
- PrimeStack_TestCaseDesign.pdf
- PrimeStack_TestExecutionReport.pdf
- PrimeStack_DefectReport.pdf
- Evidence screenshots and ZAP report files